Roteiro de Laboratório Prático: Fundamentos de Grafos e Matriz de Adjacência em C. Código em PDF.

Em dupla ou individual.

ROTEIRO DE LABORATÓRIO PRÁTICO 1

Fundamentos de Grafos e Matriz de Adjacência em C

Disciplina: Matemática Discreta
Curso: Bacharelado em Sistemas de Informação
Instituição: IFES - Campus Colatina

1. Introdução Conceitual Teórica

No estudo da Teoria dos Grafos, modelamos estruturas relacionais complexas por meio de nós (ou vértices) e conexões (arestas ou arcos).

- **Grafo Não Orientado ($G = (V, E)$):** É composto por um conjunto de vértices V e um conjunto de arestas E que estabelecem conexões **bidirecionais** (não ordenadas) entre os pares de vértices.
- **Grafo Direcionado (Dígrafo):** As arestas possuem uma orientação específica (setas), indicando um sentido de percurso de um vértice de origem para um vértice de destino.
- **Grau de um Vértice:** Representa o número de arestas incidentes a um vértice. Em *dígrafos*, o conceito desdobra-se em **grau de emissão** (quantidade de arcos que saem do vértice) e **grau de recepção** (quantidade de arcos que chegam ao vértice).
- **Casos Especiais:**
  - *Laço:* Aresta que conecta um vértice a ele mesmo.
  - *Vértice Isolado / Sumidouro:* Vértice que não possui conexões ou que possui grau zero em determinadas direções.
- **Modelagem Computacional (Matriz de Adjacência):** Para representar um grafo com V vértices em linguagem C, utilizamos uma matriz estática bidimensional `int adj[V][V]`. Adotamos a convenção binária: 1 indica a existência de uma aresta e 0 a sua ausência. Nos grafos não direcionados, a matriz possui a propriedade de simetria ($A = A^T$).

2. Descrição das Imagens para Referência Visual

Referência de Diagramas Teóricos:
• **Grafo Não Orientado vs. Simples:** Visualiza estruturas conexas, ausência de arestas e grafos simples/não simples com laços.
• **Dígrafo e Matriz de Adjacência:** Associa visualmente os arcos direcionados e o mapeamento de bits na matriz estática $V \times V$.

3. Conjunto de Exercícios de Programação em C

Abaixo estão os 4 exercícios práticos para implementação em linguagem C:

EXERCÍCIO 1.1 **Inicialização e Inserção de Arestas**

Inicializar estaticamente uma matriz de adjacência $N \times N$ com zeros e implementar uma função `void adicionar_aresta(int u, int v, int direcionado)` que marque a conexão correspondente (tratando a simetria caso o parâmetro `direcionado` seja falso).

EXERCÍCIO 1.2 **Cálculo do Grau de um Vértice**

Criar uma função `int grau_vertice(int v, int n)` que calcule e retorne o grau de um dado vértice somando os valores de sua linha correspondente na matriz de adjacência.

EXERCÍCIO 1.3 **Graus de Emissão e Recepção em Dígrafos**

Implementar as funções `int grau_emissao(int v, int n)` e `int grau_recepcao(int v, int n)` para dígrafos, verificando por meio da matriz se determinado vértice atua como um sumidouro ou elemento isolado.

EXERCÍCIO 1.4 **Verificação de Grafos Regulares**

Escrever uma função `int eh_regular(int n)` que percorra todos os vértices da matriz e verifique se todos possuem exatamente o mesmo grau, retornando 1 se verdadeiro ou 0 caso contrário.

grafo.h

```
#ifndef GRAFO_H

#define GRAFO_H


#define MAX_VERTICES 50

#define TAM_NOME 50


/**

 * @brief Inicializa a matriz de adjacência e zera o contador de vértices (Exercício 1.1)[cite: 29].

 */

void inicializar_grafo();


/**

 * @brief Obtém o índice de um vértice pelo nome ou cadastra-o automaticamente se for novo.

 * @param nome Nome textual do vértice (ex: "col")

 * @return O índice inteiro do vértice ou -1 em caso de limite excedido.

 */

int obter_ou_cadastrar_vertice(const char *nome);


/**

 * @brief Adiciona uma aresta entre dois vértices pelos seus nomes (Exercício 1.1)[cite: 29].

 * @param nome_u Nome do vértice de origem

 * @param nome_v Nome do vértice de destino

 * @param direcionado 0 para não orientado, 1 para dígrafo

 */

void adicionar_aresta_por_nome(const char *nome_u, const char *nome_v, int direcionado);
```

```
/**
 * @brief Retorna o nome textual de um vértice a partir do seu índice numérico.
 * @param idx Índice do vértice
 * @return Ponteiro para a string com o nome
 */
const char* obter_nome_vertice(int idx);

/**
 * @brief Retorna o número total de vértices únicos descobertos dinamicamente.
 * @return Quantidade de vértices cadastrados.
 */
int total_vertices_atuais();

/**
 * @brief Calcula o grau de um vértice pelo seu nome em grafos não orientados (Exercício 1.2)[cite: 29].
 * @param nome Nome do vértice
 * @param n Número total de vértices
 * @return O grau do vértice
 */
int grau_vertice_por_nome(const char *nome, int n);

/**
 * @brief Calcula o grau de emissão (out-degree) pelo nome em dígrafos (Exercício 1.3)[cite: 29].
 */
int grau_emissao_por_nome(const char *nome, int n);
```

```
/**
 * @brief Calcula o grau de recepção (in-degree) pelo nome em dígrafos (Exercício 1.3)[cite: 29].
 */
int grau_recepcao_por_nome(const char *nome, int n);

/**
 * @brief Verifica se o grafo não orientado é regular (Exercício 1.4)[cite: 29].
 * @param n Número total de vértices
 * @return 1 se verdadeiro, 0 caso contrário.
 */
int eh_regular(int n);

/**
 * @brief Exibe a matriz de adjacência com os cabeçalhos nomeados[cite: 29].
 * @param n Número total de vértices
 */
void exibir_matriz(int n);

/**
 * @brief Desenha o mapa visual ASCII com os nomes reais das cidades[cite: 29].
 * @param n Número total de vértices
 * @param direcionado 0 para não orientado, 1 para direcionado
 */
void desenhar_grafo_ascii(int n, int direcionado);

#endif
```

utils.h

```
#ifndef UTILS_H
#define UTILS_H

int ler_inteiro(const char *mensagem);
void ler_string(const char *mensagem, char *dest, int tamanho);

#endif
```

grafo.c

```
#include "grafo.h"
#include <stdio.h>
#include <string.h>

static int matriz_adj[MAX_VERTICES][MAX_VERTICES];
static char nomes_vertices[MAX_VERTICES][TAM_NOME];
static int total_cadastrados = 0;

void inicializar_grafo() {
    total_cadastrados = 0;
    for (int i = 0; i < MAX_VERTICES; i++) {
        nomes_vertices[i][0] = '\0';
        for (int j = 0; j < MAX_VERTICES; j++) {
            matriz_adj[i][j] = 0;
        }
    }
}

int obter_ou_cadastrar_vertice(const char *nome) {
    for (int i = 0; i < total_cadastrados; i++) {
        if (strcmp(nomes_vertices[i], nome) == 0) {
            return i;
        }
    }
    if (total_cadastrados < MAX_VERTICES) {
        strncpy(nomes_vertices[total_cadastrados], nome, TAM_NOME - 1);
        nomes_vertices[total_cadastrados][TAM_NOME - 1] = '\0';
        return total_cadastrados++;
    }
    return -1;
}

const char* obter_nome_vertice(int idx) {
    if (idx >= 0 && idx < total_cadastrados) {
        return nomes_vertices[idx];
    }
    return "Desconhecido";
}

int total_vertices_atuais() {
    return total_cadastrados;
}

void adicionar_aresta_por_nome(const char *nome_u, const char *nome_v, int direcionado)
{
```

```
    int u = obter_ou_cadastrar_vertice(nome_u);
    int v = obter_ou_cadastrar_vertice(nome_v);

    if (u != -1 && v != -1) {
        matriz_adj[u][v] = 1;
        if (!direcionado) {
            matriz_adj[v][u] = 1; /* Tratamento de simetria para grafos não orientados[cite: 29] */
        }
    }
}

static int grau_vertice_idx(int v, int n) {
    int grau = 0;
    if (v < 0 || v >= n) return 0;
    for (int j = 0; j < n; j++) {
        grau += matriz_adj[v][j];
    }
    return grau;
}

int grau_vertice_por_nome(const char *nome, int n) {
    int v = obter_ou_cadastrar_vertice(nome);
    return grau_vertice_idx(v, n);
}

int grau_emissao_por_nome(const char *nome, int n) {
    return grau_vertice_por_nome(nome, n);
}

int grau_recepcao_por_nome(const char *nome, int n) {
    int v = obter_ou_cadastrar_vertice(nome);
    if (v == -1) return 0;

    int grau = 0;
    for (int i = 0; i < n; i++) {
        grau += matriz_adj[i][v];
    }
    return grau;
}

int eh_regular(int n) {
    if (n <= 0) return 0;
    int grau_referencia = grau_vertice_idx(0, n);
    for (int i = 1; i < n; i++) {
        if (grau_vertice_idx(i, n) != grau_referencia) {
            return 0;
        }
    }
```

```
    return 1;
}

void exibir_matriz(int n) {
    printf("\nMatriz de Adjacencia (%d x %d):\n", n, n);
    printf("         ");
    for (int j = 0; j < n; j++) {
        printf("[%s] ", obter_nome_vertice(j));
    }
    printf("\n");

    for (int i = 0; i < n; i++) {
        printf("[%s]\t", obter_nome_vertice(i));
        for (int j = 0; j < n; j++) {
            printf("  %d   ", matriz_adj[i][j]);
        }
        printf("\n");
    }
    printf("\n");
}

void desenhar_grafo_ascii(int n, int direcionado) {
    printf("\n========================================\n");
    printf("      MAPA VISUAL DO GRAFO (ASCII)      \n");
    printf("========================================\n");

    int tem_arestas = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (matriz_adj[i][j] == 1) {
                if (!direcionado && i > j) continue;

                tem_arestas = 1;
                const char *nome_u = obter_nome_vertice(i);
                const char *nome_v = obter_nome_vertice(j);

                if (direcionado) {
                    printf("  (%s) ----> (%s)\n", nome_u, nome_v);
                } else {
                    printf("  (%s) <---> (%s)\n", nome_u, nome_v);
                }
            }
        }
    }

    for (int i = 0; i < n; i++) {
        if (grau_vertice_idx(i, n) == 0 && grau_recepcao_por_nome(obter_nome_vertice(i), n)
== 0) {
```

```
            printf("  (%s) [ISOLADO / SEM CONEXOES]\n", obter_nome_vertice(i));
        }
    }

    if (!tem_arestas) {
        printf("  [Nenhuma aresta cadastrada no grafo]\n");
    }
    printf("========================================\n\n");
}
```

utils.c

```
#include "utils.h"
#include <stdio.h>
#include <string.h>

int ler_inteiro(const char *mensagem) {
    int valor;
    printf("%s", mensagem);
    scanf("%d", &valor);

    int c;
    while ((c = getchar()) != '\n' && c != EOF);

    return valor;
}

void ler_string(const char *mensagem, char *dest, int tamanho) {
    printf("%s", mensagem);
    fgets(dest, tamanho, stdin);

    size_t len = strlen(dest);
    if (len > 0 && dest[len - 1] == '\n') {
        dest[len - 1] = '\0';
    }
}
```

main.c

```
#include <stdio.h>
#include "grafo.h"
#include "utils.h"

int main() {
    printf("=== LABORATORIO DE GRAFOS - IFES COLATINA ===\n\n");

    int tipo = ler_inteiro("O grafo e direcionado? (0 - Nao, 1 - Sim): ");
    int num_arestas = ler_inteiro("Quantas arestas (ruas) deseja adicionar? ");

    inicializar_grafo();

    char u[TAM_NOME], v[TAM_NOME];
    for (int k = 0; k < num_arestas; k++) {
        printf("\nAresta %d:\n", k + 1);
        ler_string("  Vertice (Cidade) origem (u): ", u, sizeof(u));
        ler_string("  Vertice (Cidade) destino (v): ", v, sizeof(v));
        adicionar_aresta_por_nome(u, v, tipo);
    }

    /* Descobre automaticamente o número total de vértices únicos cadastrados */
    int n = total_vertices_atuais();
    printf("\n[INFO] Total de cidades unicas cadastradas: %d\n", n);

    /* Exibe o desenho e a matriz nomeada corretamente com todas as cidades */
    desenhar_grafo_ascii(n, tipo);
    exibir_matriz(n);

    /* Testando graus */
    char vertice_consulta[TAM_NOME];
    ler_string("Digite o nome de um vertice para consultar os graus: ", vertice_consulta,
sizeof(vertice_consulta));

    if (tipo == 0) {
        printf("Ex 1.2 - Grau do vertice '%s': %d\n\n", vertice_consulta,
grau_vertice_por_nome(vertice_consulta, n));
    } else {
        printf("Ex 1.3 - Grau de emissao (saida) de '%s': %d\n", vertice_consulta,
grau_emissao_por_nome(vertice_consulta, n));
        printf("Ex 1.3 - Grau de recepcao (chegada) de '%s': %d\n\n", vertice_consulta,
grau_recepcao_por_nome(vertice_consulta, n));
    }

    /* Verificação de Regularidade[cite: 29] */
    int regular = eh_regular(n);
```

```
    printf("Ex 1.4 - O grafo e regular? %s (%d)\n", regular ? "Sim" : "Nao", regular);

    return 0;
}
```